

a specific grounding of the conclusive predicate $\hat{p}(x_1 \cdots x_r)$, it is conditioned $r! \times |\mathcal{R}|$ grounding values with the same subset of objects, of *arbitrary permutation* as the arguments to all input predicates $\mathcal{P}$. For example, consider a specific ternary predicate $\hat{p}(x_1, x_2, x_3)$. For three different objects $a, b, c \in \mathcal{U}$, the grounding $\hat{p}(a, b, c)$ is conditioned on $p_j(a, b, c), p_j(a, c, b), p_j(b, a, c), p_j(b, c, a), p_j(c, a, b), p_j(c, b, a)$ (all permutations of the parameters) for all j (all input predicates).

Our neural implementation of boolean logic rules is a lifted neural module that uniformly applies to any grounding entries $(x_1 \cdots x_r)$ in the output tensor $\hat{p}^{\mathcal{U}}$. It has a $\text{Permute}(\cdot)$ operation transforming the tensor representation of $\mathcal{P}$, followed by a multi-layer perceptron (MLP). Given the tensor representation of $\mathcal{P}$, for each $p_i^{\mathcal{U}}(x_1, x_2, \dots, x_r)$, the $\text{Permute}(\cdot)$ operation creates $r!$ new tensors as $p_{i,1}^{\mathcal{U}}, \dots, p_{i,r!}^{\mathcal{U}}$ by permuting all axes that index objects, with all possible permutations. We stack all to form a $[m^r, r! \times |\mathcal{P}|]$ -shaped tensor. An MLP uniformly applies to all m^r object indices:

$$\hat{p}(u_{i_1}, \dots, u_{i_r}) = \sigma(\text{MLP}(p_{1,1}(u_{i_1}, \dots, u_{i_r}), \dots, p_{k,r!}(u_{i_1}, \dots, u_{i_r})); \theta), \quad (2)$$

where σ is the sigmoid nonlinearity, θ is the trainable network parameters. For all sets of mutually exclusive indexes $i_1, \dots, i_r \in \{1, 2, \dots, m\}$, the same MLP is applied. Thus, the size of θ is independent of the number of objects m . This property is analogous to the implicit unification property of Horn clauses: the rule $\hat{p}(x) \leftarrow p_1(x) \wedge p_2(x)$ implicitly means, $\forall x \hat{p}(x) \leftarrow p_1(x) \wedge p_2(x)$.

Quantification. We introduce two types of meta-rules for quantification, namely *expansion* and *reduction*. Let p be a predicate, and we have

$$\textbf{(Expansion)} \quad \forall x_{r+1} q(x_1, x_2, \dots, x_r, x_{r+1}) \leftarrow p(x_1, x_2, \dots, x_r), \quad (3)$$

where $x_{r+1} \notin \{x_i\}_{i=1}^r$. The expansion operation constructs a new predicate q from p , by introducing a new variable x_{r+1} . For example, consider the following rule

$$\text{ValidMove}(x, y) \leftarrow \text{Moveable}(x) \wedge \text{Placeable}(y).$$

This rule does not fit the meta-rule in Eq. 1 as some predicates on the RHS only take a *subset* of variables as inputs. However, it can be described by using the expansion and the boolean logic meta-rules jointly.

- | | |
|---|--------------|
| 1. $\forall z \text{MoveableX}(x, z) \leftarrow \text{Moveable}(x);$ | (from Eq. 3) |
| 2. $\forall z \text{PlaceableY}(y, z) \leftarrow \text{Placeable}(y);$ | (from Eq. 3) |
| 3. $\text{ValidMove}(x, y) \leftarrow \text{MoveableX}(x, y) \wedge \text{PlaceableY}(y, x).$ | (from Eq. 1) |

The *expansion meta-rule* (Eq. 3) for a set of C r -ary predicates, represented by a $[m^r, C]$ -shaped tensor, introduces a new and distinct variable x_{r+1} . Our neural implementation $\text{Expand}(\cdot)$ repeats each predicate (their tensor representation) for $(m - r)$ times, and stacks in a new dimension. Thus the output shape is $[m^{r+1}, C]$.

The other meta-rule is for reduction:

$$\textbf{(Reduction)} \quad q(x_1, x_2, \dots, x_r) \leftarrow \forall x_{r+1} p(x_1, x_2, \dots, x_r, x_{r+1}), \quad (4)$$

where the $\forall$ quantifier can also be replaced by $\exists$. The reduction operation reduces a variable in a predicate via the quantifier. As an example, the rule to deduce the moveability of objects,

$$\text{Moveable}(x) \leftarrow \neg \text{IsGround}(x) \wedge \neg(\exists y \text{On}(y, x)),$$

can be expressed using meta-rules as follows:

- | | |
|--|--------------|
| 1. $\text{Clear}(x) \leftarrow \forall y \neg \text{On}(y, x);$ | (from Eq. 4) |
| 2. $\text{Moveable}(x) \leftarrow \neg \text{IsGround}(x) \wedge \text{Clear}(x).$ | (from Eq. 1) |

The *reduction meta-rule* (Eq. 4) for a set of C $(r + 1)$ -ary predicates, represented by a $[m^{r+1}, C]$ -shaped tensor, eliminates the variable x_{r+1} via quantifiers. For $\exists$ (or $\forall$), our neural implementation $\text{Reduce}(\cdot)$ takes the maximum (or minimum) element along the dimension of x_{r+1} , and stacks the two resulting tensors. Therefore, the output shape becomes $[m^r, 2C]$.

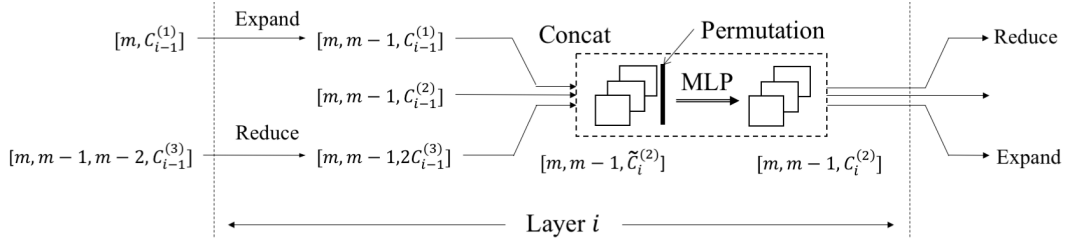


Figure 3: An illustration of the computational block inside NLM for binary predicates at layer i . $C_i^{(j)}$ denotes the number of output predicates of group j at layer i . $[\cdot]$ denotes the shape of the tensor.

2.3 NEURAL LOGIC MACHINES

NLMs realize symbolic logic rules in a multi-layer multi-group architecture, illustrated in Figure 2. An NLM has D layers, and each layer has $B + 1$ computation units as groups. Between layers, we use *intra-group computation* (Eq. 1). The predicates at each layer are grouped by their arities, and inside each group, we use *inter-group computation* (Eq. 3 and 4).

We define $\mathcal{O}_i = \{O_i^{(0)}, O_i^{(1)}, \dots, O_i^{(B)}\}$ as the outputs of layer i , where $O_i^{(r)}$ is the output corresponding to the r -ary unit at layer i . For convenience, we denote $\mathcal{O}_0 = \{O_0^{(0)}, O_0^{(1)}, \dots, O_0^{(B)}\}$ as the $\mathcal{U}$ -grounding tensors for NLM’s base predicates (the *premises*), and $\mathcal{O}_D$ at the last layer as the *conclusions*. The overall computation is performed layer-by-layer, from layer 1 to layer D . All computation units at layer i work simultaneously, taking $\mathcal{O}_{i-1}$ as inputs and generating $\mathcal{O}_i$.

Let us consider a specific group r at layer i , and we show how to calculate $O_i^{(r)}$.

Inter-group computation. As shown in Figures 2 and 3, we connect tensors from the previous layer $i-1$ in vertically neighboring groups (*i.e.* $r-1$, r and $r+1$), and aligns their shapes by *expansion* (Eq. 3) or *reduction* (Eq. 4) to form an intermediate tensor $I_i^{(r)}$:

$$I_i^{(r)} = \text{Concat} \left(\text{Expand} \left(O_{i-1}^{(r-1)} \right), O_{i-1}^{(r)}, \text{Reduce} \left(O_{i-1}^{(r+1)} \right) \right). \quad (5)$$

Nonexistent terms are ignored (e.g. when $r+1 > B$ or $r-1 < 0$). Note that from the previous layer, $O_{i-1}^{(r-1)}, O_{i-1}^{(r)}, O_{i-1}^{(r+1)}$ have shapes $[m^{r-1}, C_{i-1}^{(r-1)}], [m^r, C_{i-1}^{(r)}], [m^{r+1}, C_{i-1}^{(r+1)}]$, respectively. After the concatenation, the resulting tensor $I_i^{(r)}$ is of shape $[m^r, \tilde{C}_i^{(r)}]$, where the number of new predicates is $\tilde{C}_i^{(r)} \triangleq C_{i-1}^{(r-1)} + C_{i-1}^{(r)} + 2C_{i-1}^{(r+1)}$, and the 2 comes from the two quantifiers ($\forall$ and $\exists$).

The inter-group computation essentially aligns predicates of neighboring arities. Relational representations of different orders get combined together through the neural quantification.

Intra-group computation. The intra-group computation is implemented as the neural boolean logic in Eq. 1. It take the intermediate tensor $I_i^{(r)}$ as input, permutes and generates the output tensor $O_i^{(r)}$:

$$O_i^{(r)} = \sigma \left(\text{MLP} \left(\text{Permute} \left(I_i^{(r)} \right); \theta_i^{(r)} \right) \right), \quad (6)$$

where σ is the sigmoid nonlinearity and $\theta_i^{(r)}$ denotes trainable parameters. We apply Permute function to $\tilde{C}_i^{(r)}$ tensors in $I_i^{(r)}$ individually, and get $r! \tilde{C}_i^{(r)}$ tensors. We set the number of output neurons to be $C_i^{(r)}$, thus the shape of output tensor $O_i^{(r)}$ is $[m^r, C_i^{(r)}]$.

Example. For concreteness, in Figure 3, consider group 2 (*binary* predicates) at layer i . The module begins with the inter-group computation. It first collects the output of vertically consecutive groups (unary, binary and ternary) from the previous layer $i-1$, where their shapes are shown in the figure. Then it uses expansion/reduction to compose the intermediate tensor $I_i^{(2)}$ containing $\tilde{C}_i^{(2)} \triangleq C_{i-1}^{(1)} + C_{i-1}^{(2)} + 2C_{i-1}^{(3)}$ predicates. For each object pair (x, y) , the output $\mathcal{U}$ -grounding tensor of predicates is computed by intra-group computation $O_i^{(2)}(x, y) = \text{MLP}(\text{Concat}(I_i^{(2)}(x, y), I_i^{(2)}(y, x)); \theta_i^{(2)})$, and the output shape is $[m, m-1, C_i^{(2)}]$. The $\text{Concat}(\cdot, \cdot)$ corresponds to the Permute operation, while the MLP is shared among all pairs of objects (x, y) .

Remark. It can be verified that NLMs can realize the forward chaining of a partial set of Horn clauses. In NLMs, we consider only finite cases. Thus, there should not exist cyclic references of

predicates among rules. The extension to support cyclic references is left as a future work. See the proof in Appendix D. Thus, given the training dataset containing pairs of (*premises*, *conclusions*), NLMs can induce lifted rules that entail the *conclusions* and generalize w.r.t. the number of objects during testing.

2.4 EXPRESSIVENESS AND COMPUTATIONAL COMPLEXITY

The expressive power of NLM depends on multiple factors:

1. The *depth* D of NLM (*i.e.*, number of layers) restricts the maximum number of deduction steps.
2. The *breadth* B of NLM (*i.e.*, the maximum number of variables in all predicates considered) limits the arity of relations among objects. Practically, most (intermediate) predicates are binary or ternary and we set B depending on the task (typically 2 or 3, see Table 3 in Appendix B.)
3. The number of output predicates used at each layer ($C_i^{(r)}$ in Figure 3). Let $C = \max_{i,r} C_i^{(r)}$, and this number is often small in our experiments (*e.g.*, 8 or 16).
4. In Eq. 2, the expressive power of MLP (number of hidden layers and number of hidden neurons) restricts the complexity of the boolean logic to be represented. In our experiments, we usually prefer shallow networks (*e.g.*, 0 or 1 hidden layer) with a small number of neurons (*e.g.*, 8 or 16). This can be viewed as a low-dimension regularization on the logic complexity and encourages the learned rule to be simple.

The computational complexity of NLM’s forward or backward propagation is $O(m^B DC^2)$ where m is the number of objects. The network has $O(DC^2)$ parameters. Assuming B is a small constant, the computational complexity of NLM is quadratic in the number of allowed predicates.

3 EXPERIMENTS

In this section, we show that NLM can solve a broad set of tasks, ranging from relational reasoning to decision making. Furthermore, we show that NLM trained using small-sized instances can generalize to large-sized instances. In the experiments, Softmax-Cross-Entropy loss is used for supervised learning tasks, and REINFORCE (Williams, 1992) is used for reinforcement learning tasks.

Due to space limitation, interested readers are referred to Appendix A for details of training (including curriculum learning) in the decision making tasks, and Appendix B for more implementation details (such as residual connections (He et al., 2016)), hyper-parameters, and model selection criterion.

3.1 BASELINES

We consider two baselines as representatives of the connectionist and symbolicist: Memory Networks (MemNN) (Sukhbaatar et al., 2015) and Differentiable Inductive Logic Programming (∂ ILP) (Evans & Grefenstette, 2018), a state-of-the-art ILP framework. We also make comparisons with other models such as Differentiable Neural Computer (DNC) (Graves et al., 2016) and graph neural networks (Li et al., 2016) whenever eligible.

For MemNN, in order to handle an arbitrary number of inputs (properties, relations), we adopt the method from Graves et al. (2016). Specifically, each object is assigned with a unique identifier (a binary integer ranging from 0 to 255), as its “name”. The memory of MemNN is now a set of “pre-conditions”. For unary predicates, the memory slot contains a tuple $(\text{id}(x), 0, \text{properties}(x))$ for each x , and for binary predicates $p(x, y)$, the memory slot contains a tuple $(\text{id}(x), \text{id}(y), \text{relations}(x, y))$, for each pair of (x, y) . Both $\text{properties}(x)$ and $\text{relations}(x, y)$ are length- k vectors v , where k is the number of input predicates. We number each input predicate with an integer $i = 1, 2, \dots, k$. If object x has a property $p_i(x)$, then $v[i] = 1$; otherwise, $v[i] = 0$. If a pair of objects (x, y) have relation $p_i(x, y)$, then $v[i] = 1$; otherwise, $v[i] = 0$. We extract the key and value for MemNN’s to lookup on the given pre-conditions with 2-layer multi-layer perceptrons (MLP). MemNN relies on iterative queries to the memory to perform relational reasoning. Note that MemNN takes a sequential representation of the multi-relational data.

For ∂ ILP, the grounding of all base predicates is used as the input to the system.

3.2 FAMILY TREE REASONING

The family tree is a benchmark for inductive logic programming, where the machine is given a family tree containing m members. The family tree is represented by the following relations (predicates):